Learning Agents: From Experience to Prediction

Lecture 26

CS 4880

Kirk Duran

Logical Agents: Correct Relative to Their Knowledge

A logical agent can justify conclusions through a sound inference procedure:

$$\mathrm{KB} \vdash \alpha \quad \Longrightarrow \quad \mathrm{KB} \models \alpha.$$

KB is the knowledge base; $\vdash$ means "derivable" and $\models$ means "true in every model satisfying the knowledge base."

- This is a precise form of correctness: the conclusion follows from the stated assumptions.
- The designer must supply useful facts, rules, action models, and goals. Inference can derive actions rather than enumerate every behavior.
- Incorrect or incomplete assumptions can still produce poor real-world behavior.

Key idea

The bottleneck is specifying enough of the world. Could an agent learn some of those relationships from experience?

From Writing Relationships to Learning Them

A robot's planner needs predictions about the consequences of its actions.

- Hand-authoring a reliable rule for every operating condition is difficult.
- Actions produce observations and outcomes that can become examples.
- A learned predictor can improve the information used by an existing planner.

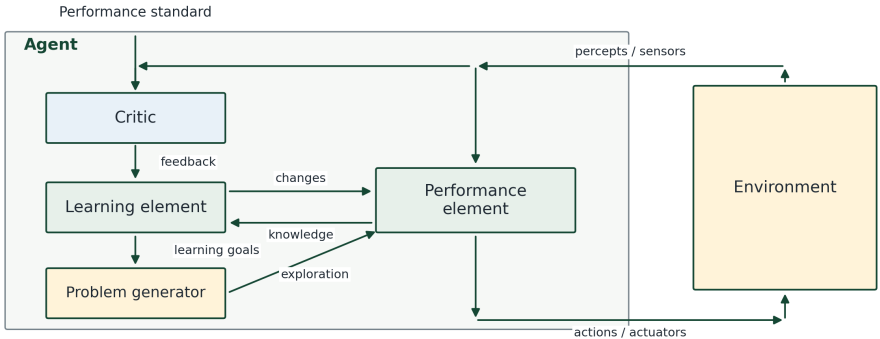
Learning and logical reasoning can coexist: learning supplies an empirical model; reasoning uses it to choose actions.



Key idea

Specify the goal and learning framework; estimate some of the relationships from data.

The Learning-Agent Loop



The agent acts, receives percepts and feedback, changes its performance mechanism, and acts again. Learning is part of this feedback loop.

What Each Part Learns or Does

- **Performance element:** chooses actions using its current knowledge and predictive models.
- **Critic:** evaluates experience against a performance standard and supplies feedback.
- **Learning element:** uses feedback to modify knowledge or model parameters.
- **Problem generator:** suggests exploratory actions that could produce useful new experience.

Key idea

Today we study supervised prediction inside this architecture: learning an input–output relationship from labeled experience. A predictor supports an agent; it is not by itself an entire agent.

A Model Approximates an Input–Output Relationship

Write an unknown numerical relationship as

$$Y = g(\mathbf{X}) + \varepsilon, \qquad \widehat{Y} = f_{\boldsymbol{\theta}}(\mathbf{X}).$$

- $\mathbf{X}$: input features; Y: observed target; g: underlying trend.
- ε: noise or effects omitted from the inputs; $\boldsymbol{\theta}$: model parameters.
- Choose a hypothesis class $\mathcal{H} = \{f_{\boldsymbol{\theta}}\}$ and use examples to find a useful member.

Key idea

Before asking how to learn a function, ask whether the chosen family can represent it. Approximation, optimization, and generalization are different questions.

Universal Approximation: An Existence Result

One standard theorem: if $g : K \to \mathbb{R}$ is continuous on a compact (closed and bounded) set $K \subset \mathbb{R}^d$, then for every $\delta > 0$ there exists a sufficiently wide, single-hidden-layer sigmoid network h such that

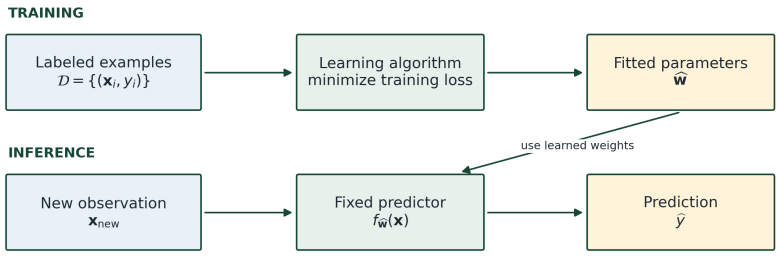
$$\sup_{\mathbf{x} \in K} |h(\mathbf{x}) - g(\mathbf{x})| < \delta.$$

- h is the approximating function; δ is a desired uniform error tolerance.
- The theorem says suitable parameters *exist*; it does not provide them or promise that finite training data will identify them.
- It does not say a straight line, a fixed-degree polynomial, or any fitted model approximates every relationship.

Key idea

Expressive models make learning plausible. They do not remove noise, optimization difficulty, or the need for held-out evaluation.

Supervised Learning: Examples Become a Predictor



$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)}_{\text{training loss}}, \qquad \widehat{y}_{\text{new}} = f_{\widehat{\boldsymbol{\theta}}}(\mathbf{x}_{\text{new}}).$$

A training example is a recorded pair $(\mathbf{x}_i, y_i)$; a model is a fitted function; a new observation supplies inputs before its target is known. Labels come from recorded outcomes or annotation.

The Same Features Can Support Different Targets

For a printer, inputs known before starting could be

$$\mathbf{x} = (\text{page count}, \text{ink coverage}, \text{queue length}, \text{temperature}).$$

Regression: one numerical value

Target $y^{(r)} \in \mathbb{R}$: completion time in minutes.
Prediction $\widehat{y} = f_{\mathbf{w}}(\mathbf{x})$.
Example loss: $(\widehat{y} - y^{(r)})^2$.

Binary classification: a category

Target $y^{(c)} \in \{0, 1\}$: completes or fails.
Estimate $p \approx P(Y^{(c)} = 1 \mid \mathbf{x})$, then assign a label using a threshold.
Example loss: binary cross-entropy.

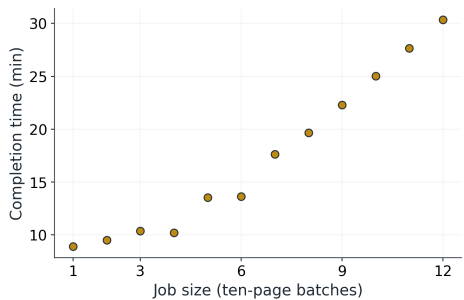
Targets and losses define different questions about the same situation. A completion timestamp is unavailable before starting: using it as an input would leak outcome information.

Supervised Learning: Predict Print Time

A print scheduler needs a duration estimate before accepting a job.

- x_i: job size in ten-page batches.
- y_i: observed completion time in minutes.
- $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$: labeled training examples; here $n = 12$.
- $\widehat{y} = f_{\mathbf{w}}(x)$: prediction from a model with parameters $\mathbf{w}$.

Training learns $\mathbf{w}$ from observed targets. Inference predicts a target for a new job.



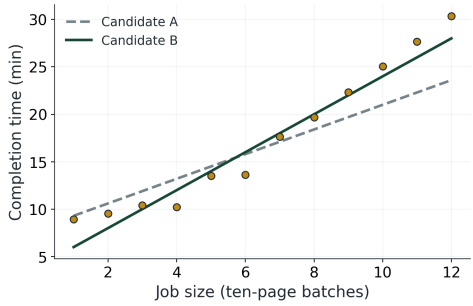
Curve Fitting: Start with a Line

Curve fitting chooses a function whose predictions match numerical observations under a specified loss. Start with

$$\widehat{y} = w_0 + w_1 x$$

- w_0: intercept, in minutes.
- w_1: slope, in minutes per ten-page batch.
- A family of lines is a **hypothesis class**: each choice of weights gives one candidate predictor.
- Which line should we prefer? We need a numerical objective.

A line assumes a constant change in predicted time per added batch.



A Python Fit: BMI and a Progression Score

Use the diabetes benchmark's BMI feature to predict a numerical progression score. Hold out the final 20 cases for this demonstration.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression

X, y = load_diabetes(return_X_y=True)
X = X[:, [2]]                  # BMI; keep a 2D feature matrix
X_train, X_test = X[:-20], X[-20:]
y_train, y_test = y[:-20], y[-20:]
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Key idea

`fit` estimates parameters using labeled training examples; `predict` applies the fixed model to new inputs. Test targets are not passed to `fit`.

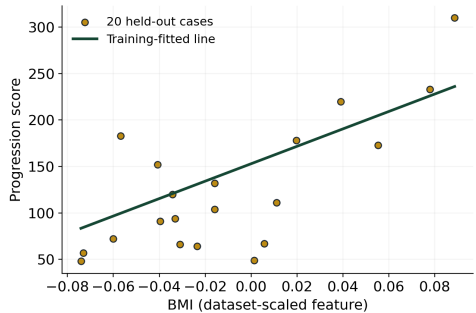
What Did `fit` Actually Learn?

On 422 training cases, the example learns

$$\widehat{y} \approx 152.919 + 938.238x,$$

where x is the dataset-scaled BMI feature.

- The model is the fitted relationship, not a copy of a single patient record.
- Points show 20 held-out outcomes; departures from the line are prediction errors.
- Software gave us coefficients. What objective selected them, and how can an algorithm find them?



Fit the Line by Minimizing Squared Error

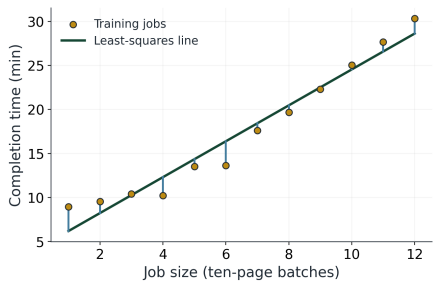
Use the residual convention predicted minus observed:

$$r_i = \widehat{y}_i - y_i = (w_0 + w_1 x_i) - y_i.$$

Ordinary least squares chooses weights to minimize

$$L(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} r_i^2, \qquad \widehat{\mathbf{w}} = \arg\min_{\mathbf{w}} L(\mathbf{w}).$$

Weights are the learned parameters; loss is the scalar objective. Squaring prevents signed errors from canceling and emphasizes large misses.



Minimizing the sum or the mean of squared errors gives the same fitted line.

The other common convention is $e_i = y_i - \widehat{y}_i = -r_i$. Since $e_i^2 = r_i^2$ and $|e_i| = |r_i|$, both conventions give the same MSE, RMSE, and MAE.

The Least-Squares Solution and a Prediction

With $\overline{x}$ and $\overline{y}$ the training-sample means,

$$\widehat{w}_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}, \qquad \widehat{w}_0 = \overline{y} - \widehat{w}_1 \overline{x}.$$

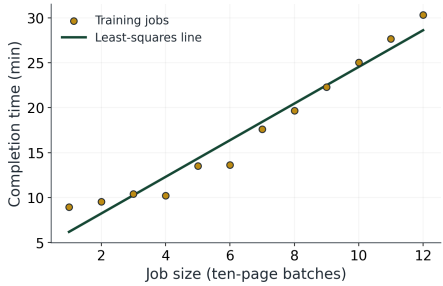
For our 12 jobs: $\overline{x} = 6.5$, $\overline{y} \approx 17.405$.

$$\boxed{\widehat{y} \approx 4.130 + 2.042x}$$

A new 60-page job has $x = 6$:

$$\widehat{y} \approx 16.38 \text{ minutes.}$$

The slope requires distinct training inputs. The intercept at $x = 0$ is outside our observed range.



How the Gradient Comes from the Loss

For $r_i = w_0 + w_1 x_i - y_i$ and $L = \frac{1}{n}\sum_i r_i^2$, the chain rule gives

$$\frac{\partial r_i^2}{\partial w_0} = 2r_i, \qquad \frac{\partial r_i^2}{\partial w_1} = 2r_i x_i.$$

Therefore

$$g_0 = \frac{\partial L}{\partial w_0} = \frac{2}{n}\sum_i r_i, \qquad g_1 = \frac{\partial L}{\partial w_1} = \frac{2}{n}\sum_i x_i r_i.$$

- Each partial derivative measures sensitivity of loss to one parameter.
- The gradient $\nabla L = (g_0, g_1)$ points in the direction of steepest local increase.
- Changing the residual convention changes derivative signs in the derivation, but not the squared-loss objective or its optimum.

Gradient Descent Updates the Whole Parameter Vector

Starting from $\mathbf{w}^{(0)}$, repeatedly evaluate the gradient and move downhill:

$$\boxed{\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w}^{(t)})}, \qquad \eta > 0.$$

Equivalently, for a line,

$$w_0^{(t+1)} = w_0^{(t)} - \eta g_0^{(t)}, \qquad w_1^{(t+1)} = w_1^{(t)} - \eta g_1^{(t)}.$$

- t counts iterations; η is the learning rate; $g_j^{(t)}$ is evaluated at the current weights.
- Compute *both* derivatives before modifying either weight: this is a simultaneous update.
- A small enough step decreases this smooth loss away from a stationary point. An arbitrary step size need not do so.

Linear least squares has a convex loss: any stationary point is a global minimum; uniqueness additionally requires sufficient independent inputs.

One Worked Update on Four Different Print Jobs

For this arithmetic example only, use $(x, y) = (1, 3), (2, 5), (3, 4), (4, 8)$. Start at $(w_0, w_1) = (0.5, 1)$ and use $\eta = 0.05$.

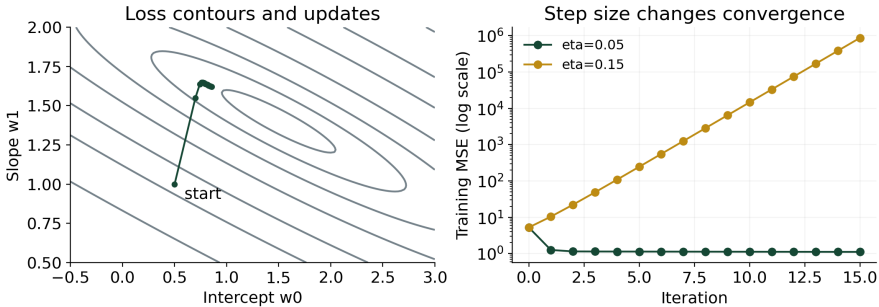
x	1	2	3	4
Observed y	3	5	4	8
Predicted $\widehat{y}$	1.5	2.5	3.5	4.5
$r = \widehat{y} - y$	-1.5	-2.5	-0.5	-3.5

$$g_0 = \frac{2}{4}(-8) = -4, \qquad g_1 = \frac{2}{4}(-22) = -11, \qquad (w_0', w_1') = (0.7, 1.55).$$

The updated predictions are $(2.25, 3.80, 5.35, 6.90)$. Training MSE drops from 5.25 to 1.25875 min^2.

This verifies one downhill step on known data. It does not establish future prediction accuracy.

Step Size Controls the Optimization Trajectory

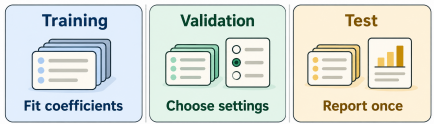


On this example, $\eta = 0.05$ moves toward a minimum; $\eta = 0.15$ diverges. Very small steps can take many iterations. Feature scale affects a useful step size.

Gradient descent is one fitting algorithm; least-squares software can instead solve the problem with numerical linear algebra.

Fitting Data and Predicting Future Experience

Reserve independent examples before fitting or choosing models.



- **Training:** fit weights; our synthetic case uses 12 jobs.
- **Validation:** compare model choices on 100 different jobs.
- **Test:** evaluate the frozen choice on 100 untouched jobs.

Our real goal is small expected loss on future examples from distribution P:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}[\ell(f(\mathbf{X}), Y)].$$

Training minimizes a sample loss; representative held-out data estimate future performance. Neither is a guarantee for every individual prediction.

Regression Metrics: How Far Are Predictions Off?

On an evaluation set of m examples, let $r_i = \widehat{y}_i - y_i$.

$$\mathrm{MSE} = \frac{1}{m}\sum_i r_i^2, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_i r_i^2}, \qquad \mathrm{MAE} = \frac{1}{m}\sum_i |r_i|.$$

Fitted line	Training	Validation	Units
MSE	2.287	1.374	minutes squared
RMSE	1.512	1.172	minutes
MAE	1.241	0.971	minutes

Smaller is better on the *same* evaluation set. RMSE emphasizes large errors more than MAE; neither bounds the error of an individual prediction.

A two-minute miss contributes 4 min^2 to squared error but 2 min to absolute error. A four-minute miss contributes 16 min^2 versus 4 min.

Relative Performance: Baselines and R^2

A simple deployable baseline always predicts the *training* mean:

$$\widehat{y}_{\text{baseline}} = \overline{y}_{\text{train}} \approx 17.405 \text{ minutes.}$$

The coefficient of determination uses the evaluation-set mean $\overline{y}_{\text{eval}}$:

$$R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \overline{y}_{\text{eval}})^2}.$$

- $R^2 = 1$: perfect predictions on this set.
- $R^2 = 0$: same squared error as always predicting this set's mean; $R^2 < 0$ is possible.
- The evaluation mean is a reference in the metric, *not* a value we use to fit our deployed baseline.
- R^2 is unitless; pair it with RMSE or MAE to judge practical error size.

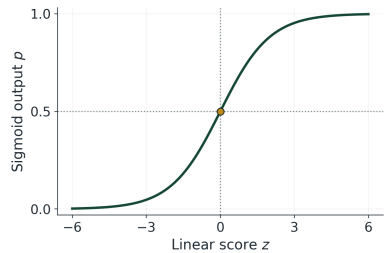
For the BMI demonstration: held-out MSE 2548.07, RMSE 50.48 score units, and $R^2 \approx 0.473$. These interpretations assume nonconstant targets.

Change the Target: Will the Printer Fail?

Print time was numerical. Now the observed label is $y = 1$ for a failed job and $y = 0$ for a completed job. Pre-job sensor readings are inputs.

$$z = w_0 + \sum_{j=1}^{d} w_j x_j, \qquad p_{\mathbf{w}}(\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- z: unrestricted linear score; d: number of input features.
- p: estimate of $P(Y = 1 \mid \mathbf{x})$, between 0 and 1.
- **Logistic regression** is a classification model despite its name.



Fit Class Probabilities with Log Loss

For a binary label $y \in \{0, 1\}$ and predicted failure probability p,

$$\ell(p, y) = -\big[y \log p + (1 - y) \log(1 - p)\big].$$

- If $y = 1$, loss is $-\log p$; if $y = 0$, it is $-\log(1 - p)$.
- For a failed job: $p = 0.8$ gives loss 0.223; $p = 0.2$ gives 1.609.
- Fit weights by minimizing average **binary cross-entropy** (log loss), using numerical optimization; software often adds a weight penalty.

Key idea

Squared error fits numerical durations; log loss fits binary class probabilities. A confidently wrong probability incurs a large log loss.

Logs are natural. For $x_{i0} = 1$, the mean-loss gradient is $\frac{\partial L}{\partial w_j} = \frac{1}{n} \sum_i (p_i - y_i) x_{ij}$: optimize the weights using the same update framework.

Evaluate Decisions with a Confusion Matrix

Predict failure when $p \geq \tau$, where τ is a threshold. A score $p = 0.7$ predicts failure at $\tau = 0.5$, but completion at $\tau = 0.8$.

100 hypothetical held-out jobs	Predict completion	Predict failure
Actual completion	64 true negatives	6 false positives
Actual failure	9 false negatives	21 true positives

- Rows are actual classes; columns are predictions. Failure is the positive class.
- A false positive triggers an unnecessary intervention; a false negative misses a failure.
- Lowering τ can catch more failures while triggering more false alarms.

Choose the threshold on validation data according to the consequences of these errors.

Accuracy, Precision, Recall, and F_1

From the preceding matrix: TP=21, TN=64, FP=6, FN=9. Failure is positive.

$$\text{accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}} = 85\%,$$

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \approx 77.8\%, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} = 70\%, \qquad F_1 = \frac{2PR}{P + R} \approx 73.7\%.$$

- **Precision:** how many predicted failures really fail?
- **Recall:** how many actual failures are detected? F_1 combines precision and recall via their harmonic mean.
- Predicting completion for every job gives 70% accuracy but zero failure recall. Accuracy can conceal failure on a rare class.

State the positive class and threshold; choose metrics according to operational costs. A missed failure and a one-minute duration error have different consequences.

A Class Probability Is Not a Correctness Guarantee

Three different evaluation questions:

- **Probability quality:** held-out log loss penalizes confident wrong predictions.
- **Decision quality:** a confusion matrix and precision/recall describe labels at a stated threshold.
- **Calibration:** among many cases assigned scores near 0.8, about 80% should actually fail if those probabilities are reliable.

Key idea

A sigmoid puts scores in $(0, 1)$; it does not prove calibration. Inspect held-out outcomes rather than treating a model score as certainty.

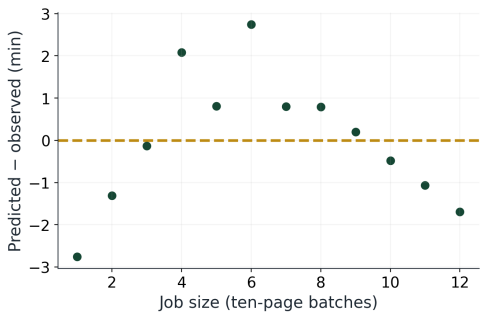
Lowering a threshold changes labels and error tradeoffs, not the underlying probability estimates. Select it on validation data.

Return to Print Time: What Does the Line Miss?

The line minimized training squared error *among straight lines*.

- Positive residual: the model predicts too much time.
- Negative residual: actual duration exceeds the prediction.
- Here residuals are negative at the ends and positive in the middle.

This curved pattern suggests a limitation of the model family. Optimization alone cannot bend a straight line.



Polynomial Regression Adds Curvature

A degree-k polynomial uses powers of the input:

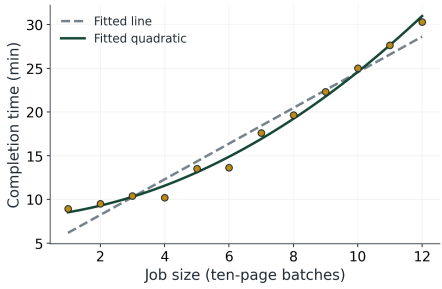
$$\widehat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_k x^k.$$

Fit a quadratic ($k = 2$) to the *same 12 jobs* by minimizing the same squared loss:

$$\widehat{y} \approx 8.028 + 0.372x + 0.1285x^2.$$

Training MSE falls from 2.287 to 0.450 minutes squared.

The marginal change is now $w_1 + 2w_2 x$: added batches need not add a constant amount of time.



Polynomial Features Keep the Least-Squares Framework

For a fixed degree k, construct $\boldsymbol{\phi}(x) = (1, x, x^2, \ldots, x^k)$:

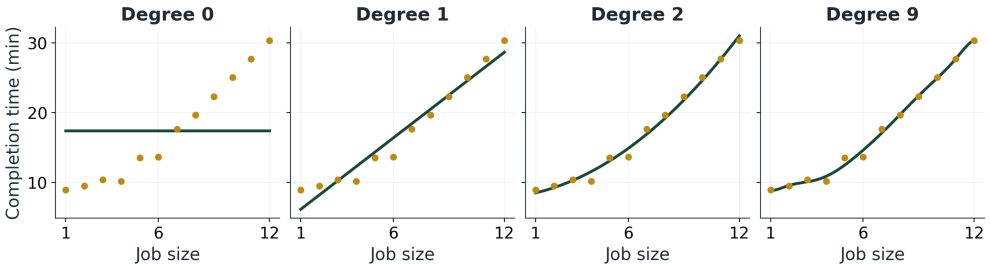
$$\widehat{y} = \mathbf{w}^\mathsf{T}\boldsymbol{\phi}(x), \qquad \widehat{\mathbf{w}} = \arg\min_{\mathbf{w}} \frac{1}{n}\sum_i [\mathbf{w}^\mathsf{T}\boldsymbol{\phi}(x_i) - y_i]^2.$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

def fit_polynomial(X_train, y_train, X_valid, degree=2):
    model = make_pipeline(PolynomialFeatures(degree, include_bias=False),
                          LinearRegression())
    model.fit(X_train, y_train)
    return model, model.predict(X_valid)
```

The curve is nonlinear in x but linear in the learned coefficients. The degree is a hyperparameter selected using validation; coefficients are fitted on training. No new test data are used to choose degree.

More Flexibility Can Fit the Sample's Noise



- A constant predicts the training mean. A line may **underfit**: too restricted to represent curvature.
- Degree 9 reduces training MSE further, from 0.450 to 0.246.
- Extra flexibility may **overfit**: follow accidental details that change in another sample.

With increasing degree, minimum training loss cannot rise: set extra coefficients to zero.

Choose Degree Using Validation Error

Each candidate is fitted on the same training set and scored on the same validation set.

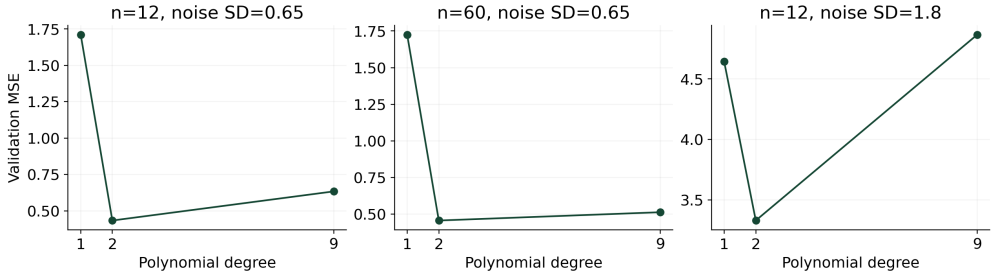
Model	Training MSE	Validation MSE
Constant (degree 0; training mean)	51.997	36.427
Line (degree 1)	2.287	1.374
Quadratic (degree 2)	0.450	**0.454**
Degree 9	**0.246**	0.701

All errors are in minutes squared. The quadratic wins on validation, although degree 9 fits the training jobs better.

Key idea

Select the quadratic on validation, then freeze it. Its untouched test RMSE is 0.614 min versus 6.420 min for the training-mean baseline.

Sample Size and Noise Change the Evidence



- More data can stabilize flexible fits; more noise makes chasing details riskier. The validation winner can change; here it stays degree 2.
- **Bias:** missing the trend. **Variance:** sensitivity to the training sample. Noise limits attainable accuracy.

Return to Classification: Two Risky Extremes

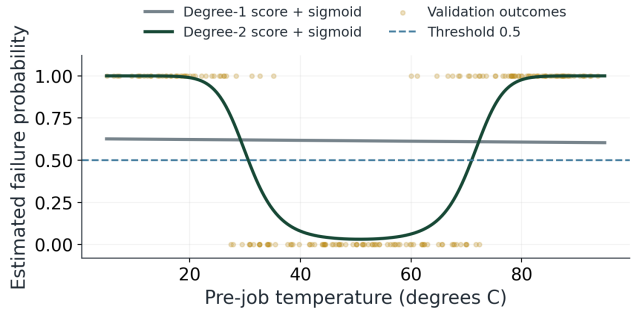
Suppose both unusually cold and unusually hot pre-job temperatures increase failure risk.

$$\underbrace{p(t) = \sigma(w_0 + w_1 t)}_{\text{linear score}} \quad \longrightarrow \quad \underbrace{p(t) = \sigma(w_0 + w_1 t + w_2 t^2)}_{\text{quadratic score}}.$$

Here t is temperature and $y \in \{0, 1\}$ is the observed failure label.

- A sigmoid of a linear score is monotonic in one input: it cannot give high failure probabilities at both ends and low probabilities in the middle.
- Adding t^2 allows that pattern. At threshold 0.5, the boundary satisfies $w_0 + w_1 t + w_2 t^2 = 0$, which can have two roots.
- Still fit using **binary cross-entropy**, not least squares on duration.

Polynomial Features Also Change a Classifier's Shape



Both models learn from the same 200 synthetic training jobs. Dots are outcomes on 200 different validation jobs; curves are estimated probabilities.

Polynomial features change the score's shape; the sigmoid, binary target, log-loss objective, and threshold rule remain the same.

Evaluate the More Flexible Classifier Again

Compare validation log loss, then freeze the choice and threshold before using the test set.

Model score	Validation log loss	Validation accuracy
Linear	0.692	56.5%
Quadratic	0.204	91.5%

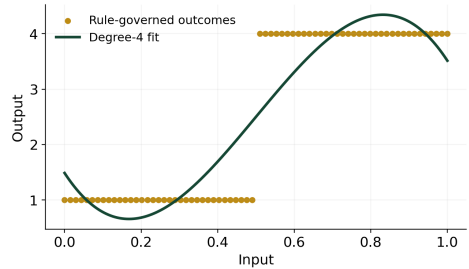
The quadratic is selected by lower validation log loss. On 200 untouched test jobs, its log loss is 0.243 and accuracy is 94.0% at $\tau = 0.5$.

- Here the test set contains 3 false positives and 9 false negatives; accuracy alone can hide an expensive error type.
- Polynomial regression uses squared loss for a number; logistic regression with polynomial features uses log loss for a class probability.
- Degree and threshold choices belong to validation, just as degree selection did in regression.

When a Smooth Fit Is the Wrong Representation

Some systems have operating modes, abrupt transitions, or interactions between categorical and numerical features.

- A low-degree curve can smooth over a meaningful jump.
- Increasing degree may add oscillation or numerical instability rather than useful structure.
- Continuity prevents arbitrarily small uniform error across a genuine jump.



The theorem assumes continuity. A good statistical fit also does not establish causal rules; better features or a different model family may be needed.

Make Evaluation Match the Intended Use

- **Available inputs:** a completion timestamp would leak the duration. Use only information available when making the prediction.
- **Preprocessing:** learn scaling or imputation values on training data; reuse them unchanged on validation and test inputs.
- **Relevant splits:** for future jobs, consider a time-based split; to predict on new printers, keep each printer's records together.
- **Coverage:** our duration models saw $x \in [1, 12]$. Good errors there do not validate polynomial extrapolation to much larger jobs.
- **Uncertainty:** scores from a finite test sample vary. Keep the test set out of repeated tuning.

Key idea

The same evaluation discipline applies to numerical predictions and categorical decisions, with metrics suited to each target.

From Deductive Guarantees to Empirical Evidence

1. Logical reasoning derives conclusions from an explicit knowledge base; learning can estimate relationships that are difficult to specify by hand.
2. Labeled experience defines inputs, targets, a hypothesis class, and a loss. The agent's learning element fits the model used for future predictions.
3. Least squares and gradient descent fit numerical models. Logistic classification changes the output interpretation and training loss.
4. Polynomial features increase representational flexibility for both tasks, while increasing the opportunity to fit noise.
5. Training loss measures fit; representative held-out data and task-appropriate metrics supply evidence about future usefulness.

Key idea

Approximation asks what can be represented; optimization asks what can be fitted; generalization asks what transfers to new experience.